AI Agent Knowledge Base for a Financial Company

Status: Draft (pending approval) · Date: 2026-08-27

Contents

The Problem Nobody Wants to Own

Somewhere in the firm's file shares, SharePoint sites, mail archives, and document management systems sit roughly **three million documents**: credit agreements, amendments, side letters, board decks, risk memos, spreadsheets, and thirty years of scanned paper. Somewhere in there is the answer to almost any question a lawyer, credit officer, or risk analyst might ever ask.

Nobody can find it.

Keyword search cheerfully returns four hundred hits, which is a polite way of returning zero. The one person who knew which document actually mattered retired in the spring. And the version someone finally digs up? Superseded two amendments ago — a fact that will surface at the worst possible moment, because that is when such facts surface.

So the ask sounds simple: *build an AI assistant that answers questions from our documents.* Lovely. Except this is a regulated financial firm, so the real ask comes with fine print:

- Answer from the documents, and **prove it** — citations down to the page, tied to the exact version used. "Trust me" is not a citation.
- Show a user **only what they're entitled to see**. Not one chunk more. Ethical walls between deal teams are law, not office etiquette.
- Keep everything — documents, indexes, models, logs — **inside the firm's walls**. Nothing takes a field trip to someone else's cloud.
- Keep a **tamper-evident record** of every question and answer, because a regulator may ask about it years from now, and "we're not sure what it said" is a career-limiting answer.
- And when the system doesn't know, it must **say so** — not improvise something plausible in a confident voice.

That last point deserves a moment of silence. A chatbot that hallucinates a covenant threshold isn't a productivity tool; it's a liability engine with a friendly interface. Everything in this design flows from taking that seriously.

One deliberate act of modesty: version 1 is **read-only**. It answers questions. It executes no trades, sends no emails, makes no decisions. Walk before you run — especially in a building full of compliance officers.

The Temptation of Shiny Infrastructure

The first challenge isn't technical. It's saying no.

The standard architecture for a system like this — the one in every vendor deck — involves a vector database, a search cluster, a message broker, a workflow engine, and a graph database. Collect all five! Each one is a distributed system to operate, patch, secure, and keep synchronized with the others. Every pair of systems is a place where data can quietly disagree. Every extra cluster is a pager that will eventually go off at 2 a.m., and it will not be sorry.

This is how projects die, by the way. Rarely from missing features — almost always from drowning in their own plumbing, sometimes before the first user asks the first question.

So this design adopts **simplicity as its guiding principle**: the simplest, most elegant architecture that still delivers the full functionality — the minimum number of moving parts, each one chosen on purpose. Simplicity is what makes the project *do-able* — and flexible, maintainable, and affordable. We can prototype fast, put something real in front of users in weeks, and run the whole thing without a big team, a fleet of clusters, or exotic hardware.

And here's the delicious part: the numbers say we can get away with it. Our corpus — big as it feels — produces about **20–25 million searchable chunks**. That is not "big data." For a well-fed PostgreSQL instance, that's a Tuesday.

So the solution is a bet on gloriously boring technology: one PostgreSQL cluster holds everything transactional — the document registry, the entitlements, the search indexes (vector *and* keyword), the job queue, and the audit log. The team already knows how to run PostgreSQL. Access control becomes a SQL join

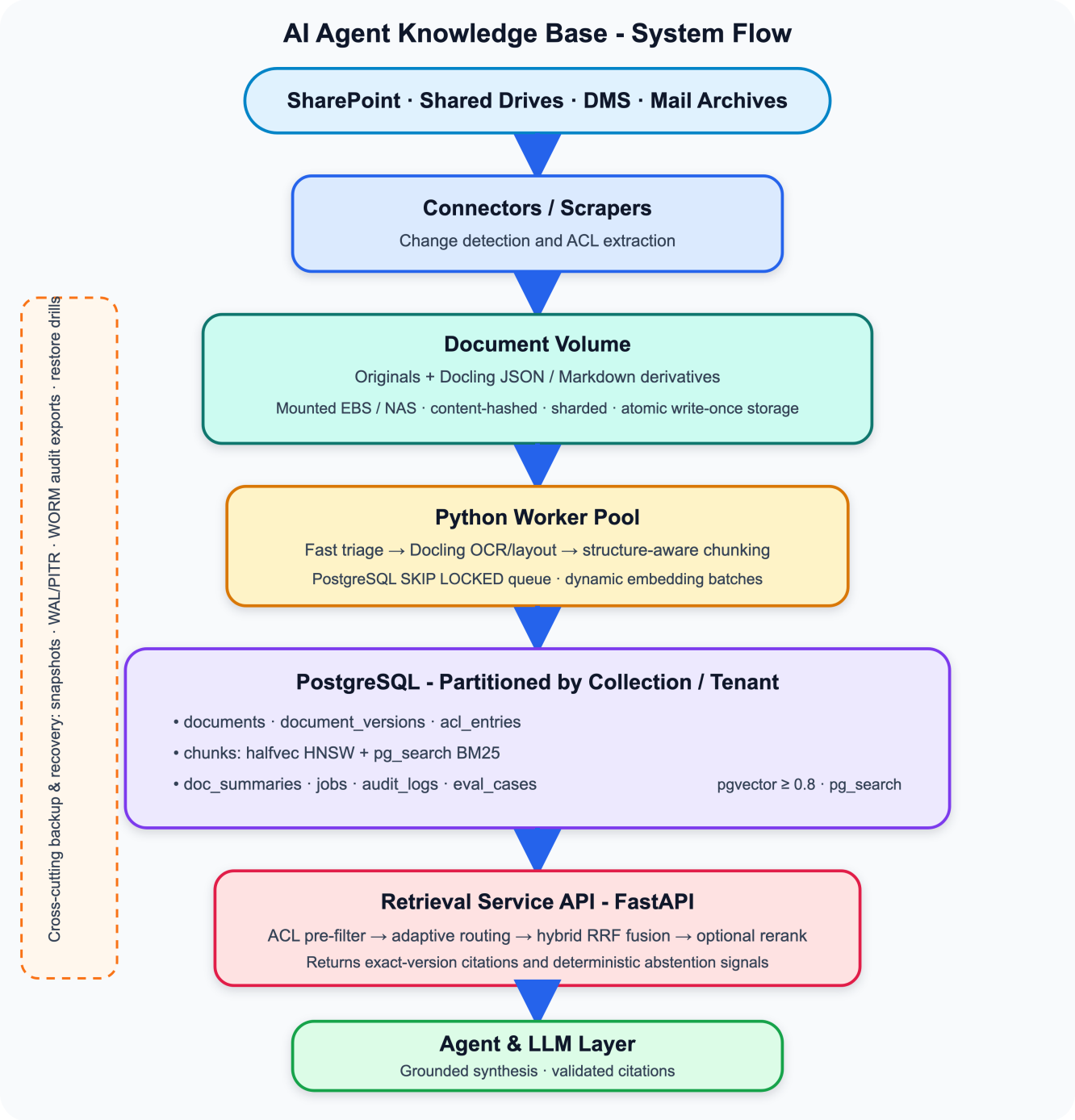
instead of a distributed-systems research project. There is exactly one source of truth, and everyone knows its address.

Because bets should be falsifiable, the design writes down **scaling gates** — measurable thresholds (chunk count above 50 million, p95 latency above 2 seconds despite tuning, index churn degrading reads) at which we'd graduate to specialized infrastructure. Until a gate trips, we don't. When one does, the migration is a background re-index, not a crisis — for reasons that will become clear shortly.

The same restraint picks the language: Python, not Rust. The question comes up in every design review, so let's answer it here. A Rust rewrite makes the code faster — but look at where this system actually spends its time: search runs inside PostgreSQL (written in C), embeddings and reranking run on GPUs (CUDA kernels), PDF parsing runs in PyMuPDF (C bindings) and Docling's native internals. The Python code is a thin orchestration layer — it moves job rows, calls libraries, and serializes API responses. Rewriting the glue in Rust makes the glue faster, and the glue was never the bottleneck.

Meanwhile the costs would be real. The document-understanding and ML ecosystem is Python-first — Docling *is* Python, and the best parsing, OCR, and model tooling all live there. The team knows Python; prototypes ship in days, and the pool of people who can maintain the system stays large. Choosing Rust here would mean either wrapping the same Python/C libraries in FFI (all the complexity, none of the benefit) or abandoning the best tools for the job — trading the crown jewels for type safety in the plumbing.

And modularity keeps even this decision reversible: if some module ever proves CPU-bound under *measurement* — chunking or deduplication at a much larger scale, say — that one module can be reimplemented in Rust behind the same contract, invisible to everything else. No crisis, no rewrite; a swap. Though the honest first answer to a slow worker is usually even more boring: add another worker node. Hardware is cheaper than a second language in the team's stack.



Don't Let the Dumplings Fuse

Simplicity has an evil twin, and it shows up wearing simplicity's clothes.

Think of a good system as a plate of dumplings. There's nothing wrong with a dumpling — a dumpling is a small masterpiece of engineering. Each one has a wrapper that keeps its filling to itself (*encapsulation*), each one holds its own distinct filling (*separation of concerns*, a *bounded context* in a doughy jacket), each can be

cooked and tasted on its own (*developed and tested separately*), and they sit loosely together on the plate, touching but never sticking (*loose coupling*). That is exactly the architecture we want.

The failure mode is what happens when you boil them too long and walk away. It goes like this. A small team builds a lean system — one database, a handful of components, nothing wasted. Then, under deadline, the agent code starts querying the chunks table directly (why bother with an API for an internal tool?). The parser learns which embedding model is in use (it was convenient). Every shortcut is individually reasonable. Two years later the dumplings have **fused into a single messy clump**. The diagram still shows few components — but you can no longer change *any* of them without understanding *all* of them, and nothing can be developed or tested on its own anymore. The system is small, and it is stuck.

That failure mode matters doubly here, because this design makes bets — one database for everything, self-hosted models, a specific parser — and bets must stay reversible. The scaling gates promise "when a threshold trips, we migrate calmly." That promise is only honest if the thing behind the gate can be swapped without a rewrite.

So the second guiding principle, alongside simplicity, is modularity: every capability owns its domain and is used only through its contract.

- **Retrieval is a service, not a schema.** Agents and internal systems talk to the FastAPI retrieval API; not one of them knows a table name. The day the chunk index moves to a dedicated search cluster, consumers won't even notice.
- **Connectors are plugins.** Each source system speaks its own dialect on one side and emits the same normalized events — file, metadata, ACL — into the same job queue on the other. Adding a source means writing an adapter, not performing surgery on the pipeline.
- **Parsers hide behind the derivative contract.** Docling and PyMuPDF both produce the same structured JSON + Markdown shape. A better parser next year changes nothing downstream.
- **Models are versioned data, not wiring.** Every chunk records which embedding model produced it; upgrades run blue/green as a parallel index and cut over only when evaluation approves. The reranker and the LLM sit behind their own interfaces, equally swappable.
- **Dependencies point one way.** Sources are permanent; derivatives derive from sources; chunks, indexes, graph, and wiki derive from derivatives. Nothing downstream ever writes upstream. Water flows downhill only.

The same discipline reaches into the code itself, because a module with beautiful external contracts can still rot from within — one 3,000-line file, functions that scroll like film credits, and not a word of explanation anywhere. That's the dumplings fusing again, one level down:

- **Small files:** nothing longer than **800 lines**. A file past that is holding more than one responsibility and should be split — it's not a file anymore, it's a hostage situation.
- **Small functions:** nothing longer than **50 lines**. A function that doesn't fit on one screen is several functions in a trench coat.
- **Docs at every level:** each file opens with a doc block stating its purpose; every function and class carries its own. Code explains *how*; docs explain *what for*.
- **Directories as modules:** code splits into subdirectories along module boundaries, and **each directory has its own README.md** — what it owns, its public API, what it depends on. A newcomer should be able to parachute into any directory and know where they've landed.

The test for every boundary — architectural or code-level — is blunt: **can this piece be changed, replaced, or removed without understanding the whole system?** If not, the seam is in the wrong place. Simplicity keeps the number of parts small; modularity keeps each part replaceable. This design insists on both.

Where Do Three Million Files Live?

Databases make terrible file cabinets, and cloud object storage is off the table — the compliance boundary says everything stays on infrastructure the firm controls. So originals live on **mounted volumes** (AWS EBS behind an NFS export, or the on-prem NAS), and the database stores only a `storage_uri` pointing at each file. The database knows *where* everything is; it just refuses to carry it.

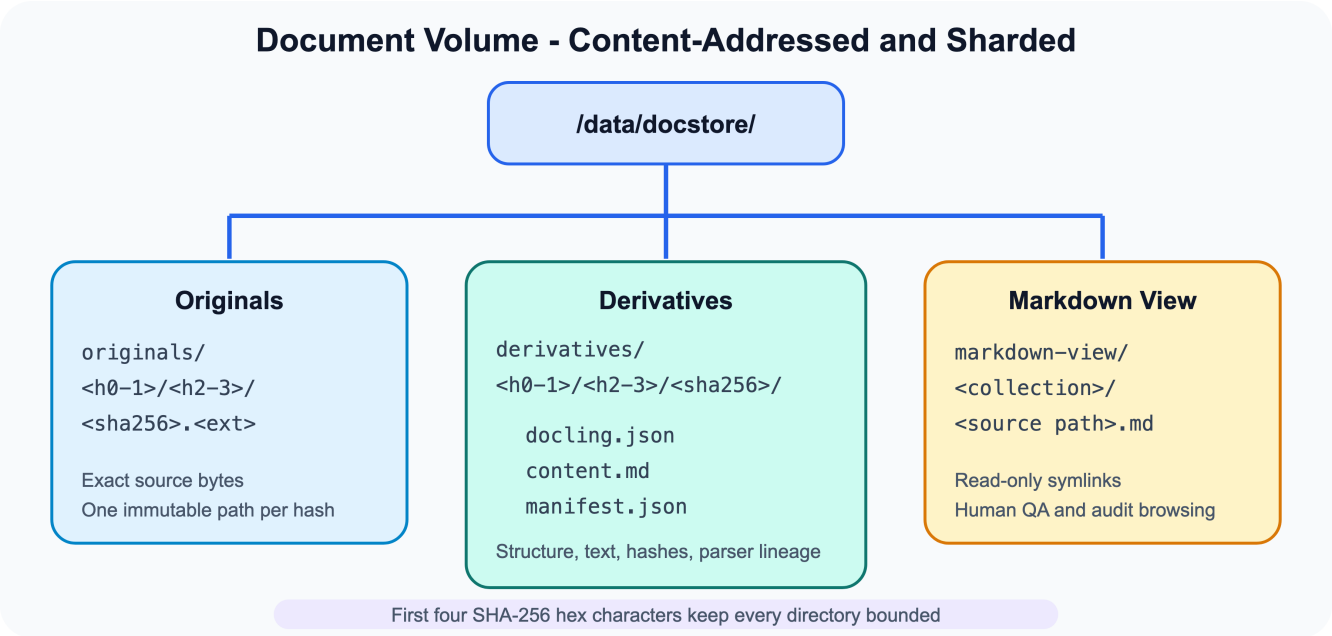
That sounds mundane until you ask what three million files do to a filesystem. Dump them into one directory and every listing crawls — that's how you make a filesystem cry. Let people overwrite files in place, and you can never again prove what a citation pointed to.

Two old-fashioned disciplines solve both:

- **Content addressing.** Every file is stored under its SHA-256 hash, sharded into subdirectories by the first four hex characters (`originals/ab/cd/<sha256>.pdf`), so no directory ever holds more than a few thousand entries. Same bytes, same path, forever; a changed document gets a *new* hash and a *new* path.
- **Write-once.** Workers write to a temp path, `fsync`, then atomically `rename()` into place. Nothing is ever overwritten. Once written, a file is immutable — optionally sealed with `chattr +i`, so even a root process can't quietly rewrite history.

This buys something quietly wonderful: **a snapshot of the volume is consistent by construction**, at any instant, no coordination required — a half-written file exists only under a temp path that nothing references. File that fact away; it becomes the hero of "The Day Everything Goes Wrong."

Alongside each original live its **derivatives** — the parsed, normalized representations (structured JSON plus clean Markdown). And because storage can lie slowly and silently, a weekly scrub job re-hashes samples and compares them against the database, hunting bit rot.



Capacity is comfortable: ~3 TB of originals, 1–2 TB of derivatives; provision 8–10 TB and grow with LVM.

Teaching the System to Read

The corpus is a museum of formats: born-digital PDFs, Word contracts, PowerPoint decks where the real story hides in the speaker notes, spreadsheets with opinions, and scans of paper signed before some of our analysts were born. Run everything through a heavyweight OCR pipeline and the backfill takes months. Run everything through a fast text extractor and it butchers the scans and shreds every table.

The solution is triage. A fast path (PyMuPDF) inhales born-digital files at 50–100 pages per second per node. A heavy path — GPU workers running Docling with layout analysis and OCR — takes the hard cases at 8–15 pages per second: scans, brutal multi-column layouts, decks. Every scanned page gets an **OCR confidence score** that follows its text all the way into answers: a citation built on shaky OCR admits it, right there in the citation.

Two principles govern the pipeline:

Parse once, index many times. Parsing three million files costs weeks of compute, and we refuse to pay twice. Parsed derivatives are stored permanently. Re-chunking, re-embedding, upgrading models, even defecting to a different search engine — all of it reads from derivatives, never from raw files. This is precisely what makes the scaling gates cheap to cross: the entire search layer is *disposable*.

Deduplicate before indexing. The same memo lives in five SharePoint sites and eleven inboxes, because that is how offices work. Exact duplicates get caught by hash before they're even fetched twice; near-duplicates by MinHash/SimHash on normalized text. Aliases map every copy to one canonical version. The arithmetic is satisfying: ~45 million potential chunks collapse to 20–25 million indexed ones — half the index we'd otherwise be paying to search.

The whole pipeline runs as idempotent steps in a transactional job queue — plain PostgreSQL rows claimed with `SKIP LOCKED`, no message broker in sight. A crashed worker resumes exactly where it stopped. A re-delivered event is a shrug and a no-op.

The Art of Breaking Documents Apart

Search engines don't retrieve documents; they retrieve *chunks*. Chunk badly and everything downstream suffers. Split a contract mid-sentence every 500 tokens and you get fragments that match queries but can't support answers. Treat a financial table as prose and its meaning evaporates — a number without its header row isn't data, it's confetti.

The solution: cut along the document's own seams.

- **Narrative documents** split at heading boundaries, 400–800 tokens, on clean paragraph breaks. Each chunk carries its full heading path — `Credit Agreement > Section 4.2 > Representations` — so it never forgets where it came from.
- **Slides:** one slide per chunk, with deck title, slide title, and presenter notes together. (In decks, the notes are where people write what they actually think.)
- **Tables:** kept whole, or split as row-groups that each repeat the header row; the raw structured version rides along in metadata for exact numeric lookups.
- **Scans:** chunked like narrative, wearing their OCR confidence; anything below 85% gets flagged in prompts and citations.

One principle rules the data model: **documents are not chunks.** A document has lifecycle, lineage, entitlements, and legal retention — it's a citizen. A chunk is a disposable search artifact, rebuildable at any time from derivatives. Confusing the two is how systems end up unable to delete what the law requires them to delete — an awkward conversation nobody wants to have.

Three Ways to Find a Needle

Now the heart of the matter. Users ask two fundamentally different kinds of questions, and each one breaks the search method built for the other:

- "*What is our aggregate exposure to commercial real estate in Europe?*" — thematic, conceptual. Keyword search is useless; no document contains that sentence.
- "*Find clause 7.3(b) of the Meridian facility agreement.*" — an exact needle. Semantic search is actively dangerous here: it will happily return something *similar* to clause 7.3(b), delivered with total confidence. Similar is worse than nothing.

Solution one: run both searches, always. Semantic search (vector embeddings, computed by self-hosted models on our own GPUs) catches meaning and paraphrase. Keyword search (BM25 full-text) catches tickers, ISINs, clause numbers, party names — the stuff lawyers actually type. Results merge by reciprocal rank fusion, and a cross-encoder reranker picks the best handful.

Solution two: search at two altitudes. Twenty-five million chunks is a noisy haystack for a broad question. So every document also gets a one-paragraph **summary with its own embedding** — a three-million-card catalog.

Broad queries consult the catalog first, shortlist the promising documents, then search chunks only within them. Needle queries — anything with an exact identifier — skip the catalog and hit the full chunk index directly, because a summary might not mention clause 7.3(b), and a missed clause is a failed audit.

Solution three: after matching, widen the lens. Chunks are sized for *matching*, but the sentence that matches a query and the sentence that answers it are frequently next-door neighbors. So each selected chunk gets expanded to its parent section — its siblings by heading path, within a token budget — before the model reads it. The match finds the spot; the section supplies the meaning.

And beneath all of it, without exception, sits the entitlement check. **ACL filtering happens in SQL, before ranking.** An unauthorized document doesn't rank low — it doesn't exist. The permission-leak tolerance is written down as a number, and the number is **0.00%**.

The Web Between the Documents

A few weeks into any project like this, someone asks the question that vectors and keywords simply cannot answer: "*Which amendments modify this credit agreement?*"

Similarity search finds text that *resembles* other text. But an amendment doesn't resemble the agreement it modifies — it *references* it, tersely, by title and date, the way legal documents nod to each other across the room. Those relationships — supersession, amendment, exhibit, citation, same deal — are the corpus's skeleton, and text search is completely blind to it.

Here's the lucky break: legal documents *announce* their relationships. "This Amendment No. 3 to the Credit Agreement dated March 15, 2024…" is a database row wearing a costume. No AI required — regexes and parsers pluck these cross-references out during ingestion, along with the entities (parties, ISINs, deal names).

So the design adds a document graph, in plain PostgreSQL tables: typed edges (`amends`, `supersedes`, `exhibit_of`, `references_clause`, `cites`) between documents, plus an entity index recording who is mentioned where. Every edge remembers how it was extracted and with what confidence — no anonymous edges.

The graph earns its keep twice:

At retrieval time, after hybrid search returns its chunks, the engine walks one hop. Retrieve a credit agreement, and its amendments come along — even though they never matched the query. Retrieve something with a `supersedes` edge pointing at it, and the answer gets *flagged as superseded* — structurally, not by hoping the newer version happened to rank well. And every hop joins the ACL table first: a link must never reveal so much as the *existence* of a document the user can't see.

For humans, the graph renders as an **interconnected wiki**: one Markdown file per document, YAML frontmatter on top (type, dates, classification, status), `[[wikilinks]]` for every relationship — browsable in Obsidian, greppable with ripgrep, laid out in a readable tree (`wiki/documents/<collection>/<type>/<year>/…`, plus a stub page per entity listing everywhere it appears). Compliance officers get to navigate the corpus by its actual structure instead of playing search-term roulette.

One rule keeps the wiki honest: **it is a projection, not a source.** Nobody edits wiki files — they're rendered nightly from the database and can be deleted and regenerated without losing a byte of truth. And since files on disk have no row-level security, the full wiki renders only inside the curators' enclave. Links leak existence, and around here, existence is confidential too.

Later (v2), community detection over this graph will cluster the corpus into themes with summary pages — the corpus explaining its own neighborhoods.

What Was True Last March?

Sooner or later — probably sooner — someone will ask: "*What was the covenant threshold as of Q3 2024?*"

A system that only knows "current vs. superseded" answers with today's amendment: fluently, confidently, and wrongly. In finance, *when* something was true is half the fact.

The solution is to give time a seat at the table. Every document version carries an effective window (`effective_from` / `effective_to`); every graph edge carries a validity window. The dates come from the documents themselves — effective-date clauses are dated boilerplate, extractable with a regex and a straight face — and from lineage: creating a supersession edge automatically closes the predecessor's window.

On top of that:

- The retrieval API accepts an `as_of_date`. Instead of filtering to current versions, it filters to versions *in force on that date* — and the answer is clearly flagged as historical, so nobody mistakes last year's covenant for this year's.
- When retrieved passages disagree *because they come from different eras*, the system doesn't wring its hands about a "contradiction" — it lays out a **timeline**: "the threshold was X from March 2024, amended to Y effective November 2024." Not a bug; a biography.
- Ranking gets a sense of time: queries with date intent boost chunks near that date; queries without it get a gentle nudge toward in-force versions, so stale text can't out-charm live text on similarity alone.

The Corpus Never Sits Still

Everything so far described a system at rest. Real corpora don't rest: new documents arrive, versions supersede, deletions happen, and — most dangerous of all — someone's access gets revoked. Each event must ripple through every layer we've built: registry, chunks, vectors, indexes, summaries, graph edges, wiki pages. Miss one layer, and the layers start lying to each other.

The solution is a defined lifecycle with three hard rules.

When a document is **added**, it flows through the standard pipeline and is searchable within minutes. When one is **updated**, the new version gets fresh chunks and a `supersedes` edge; the old version's chunks are *deactivated, never deleted*. That's rule one — **soft-deactivation** — because an audit-log citation from last year must still resolve to the exact text the user saw, for as long as the records policy says. When a document is **removed** at the source, it's tombstoned: invisible to retrieval the instant the transaction commits (the active-flag and ACL filters run before ranking, so there's no index rebuild to wait for), while the underlying files quietly ride out their retention period.

Rule two: **the visibility flip is atomic.** Old chunks off, new chunks on, current-flag moved — one transaction. No query ever catches a document half-dressed.

Rule three: **ACL changes outrank everything.** A revoked entitlement propagates within **five minutes**, monitored as a first-class alert, with a daily full re-sync to sweep up anything a missed webhook dropped. Stale content is embarrassing; stale permissions are a breach.

The slower layers refresh on their own schedule — wiki re-renders and link re-extraction nightly, cluster refresh and physical garbage collection weekly to monthly. And because deactivated vectors pile up as dead weight in the vector index, a weekly recall check compares the index against a brute-force scan on a sample; if recall sags, that partition gets rebuilt during a maintenance window. Even indexes need a gym membership.

Trust, but Verify

This system changes in two ways, and both can break it without making a sound. Engineers change the **code** — and no compiler complains when someone imports the database driver into the domain logic or bypasses the retrieval API "just this once." The **corpus** changes daily — and RAG systems degrade quietly while hallucinating confidently: a connector dies on a Tuesday and nobody notices until someone asks about a

document from the gap; an embedding upgrade shuffles rankings and recall drops three points with no error message anywhere. Silence, it turns out, is not a good sign. Silence is just silence.

So verification runs at both timescales: every code change passes a pyramid of tests before it merges, and every day's corpus changes pass an evaluation gate before the night is out.

Guarding the code

Rules that nothing enforces are wishes. And untested code develops a second disease: nobody dares touch it. A codebase that can't be safely changed is the fused dumpling clump's final form — frozen solid. So every change runs the gauntlet:

- **Unit tests** guard the functions: fast, isolated, no network, no database, dependencies faked at the port interfaces — which is only possible because the ports exist. Modularity pays its first dividend here.
- **Module tests** guard the contracts: each module — connectors, parsing, chunking, retrieval, graph, wiki renderer — is tested in isolation through its **public API**, dependencies stubbed. If you can't write a module's tests without reaching into another module's internals, congratulations: you've found a boundary in the wrong place, cheaply.
- **Integration tests** guard the real seams: SQL against an actual PostgreSQL in a disposable container (pinned by digest, naturally), and the pipeline end-to-end against a fixture corpus — a scanned PDF, a deck, a spreadsheet, a near-duplicate pair — confirming that ingestion, chunking, embedding, and retrieval still agree with each other.
- **Architecture-conformance tests** guard the rules themselves. The mechanical rules go to mechanical tools: `import-linter` fails the build if domain code imports infrastructure; file-length, function-length, and docstring checks run in CI, tirelessly and without opinions. But some rules need *judgment* — is this new dependency direction legitimate? is this "utility module" quietly becoming a junk drawer? For those, **an AI agent reviews every change against the written architecture rules**, flags violations with the specific rule it believes is broken, and gates the merge like any failing test. Humans adjudicate disputes; the agent just makes sure erosion never happens *silently*.

Guarding the answers

Code tests prove the machine works. A separate discipline proves it's still *right*. The yardstick is a **golden test suite**: 300+ real questions curated with Legal, Risk, Credit, and Operations, with known correct answers *and known correct citations*. It includes as-of temporal cases, adversarial permission-leak attempts, and — the fun part — an **unanswerable set**: questions whose answers are deliberately absent, locked behind entitlements, or found only in superseded text. The correct response is abstention. A confident answer to an unanswerable question is a hallucination caught red-handed.

But where do test questions and grading criteria come from? The tempting shortcut is a conference-room checklist — "answers should be accurate, relevant, and clear, scored 1 to 10." Checklists born this way fail twice: the scores are too mushy to act on, and the criteria only cover failures somebody predicted. The system's actual failures have more imagination than that.

So the evals are grown from real outputs, not abstract principles. During the pilot, evaluation starts with actual **traces** — the query, the routing decision, every chunk retrieved with its scores, the final answer — put in front of domain experts in a review interface where they annotate problems *in context*: the sentence that overstates, the citation that doesn't support its claim, the abstention that should have been an answer. **Failures**

get reviewed before criteria get written. AI helps with the labor — clustering similar failures, surfacing patterns, drafting rubric items — but humans supply the taste: only Legal knows which paraphrase of a covenant is materially wrong.

From those annotated traces, criteria emerge along two paths:

- **Top-down criteria** encode what the task always required: every claim carries a citation; the abstention phrasing is exact; nothing in the answer comes from outside the retrieved context; format and policy hold.
- **Bottom-up criteria** capture what the traces actually revealed — patterns no whiteboard session would have produced: "*quotes the superseded threshold when both versions were retrieved,*" "*merges two counterparties with similar names,*" "*answers from general legal knowledge instead of the documents.*"

Every criterion becomes a **specific pass/fail check, never a vague score** — not "faithfulness: 7/10" but "every numeric claim appears in a cited chunk: yes or no." Binary checks are debuggable: a failure points at exactly one thing to fix. And when the checks are automated with LLM judges, **each criterion gets its own judge pass** — one mega-prompt asked to verify twelve requirements will quietly forget a few; twelve small judges with one question each forget nothing.

Then the flywheel closes: every validated failure pattern graduates into permanent machinery — a regression case in the golden suite, a CI check that runs before any prompt or model change ships, a dashboard series, a production monitor. The suite is the accumulated memory of every way the system has ever been caught being wrong. It only grows.

The nightly gate

Every night, after the day's sync settles, the full suite runs against **production** indexes:

- Results compare against a rolling 7-day baseline. Recall down more than 2 points? Citation precision slipping? Abstention correctness off? **Any** permission leak at all? The gate trips: on-call gets paged and ingestion freezes until a human understands why. A failed eval is treated exactly like a failed deploy — because that's what it is.
- A **reconciliation count** runs across three layers — source systems → registry → active chunks — so a silently dead connector shows up as a number that doesn't match *tonight*, not as a mystery at quarter-end.
- A **self-retrieval probe** samples documents added or changed that day, generates a query from each one's own content, and verifies the document actually comes back. Proof that new content is *findable*, not merely stored somewhere warm.

And because golden questions can't cover real traffic, hallucination gets a continuous watch: a **self-hosted judge model** samples production answers daily and checks entailment — does each cited chunk actually support the claim leaning on it? Abstention rates are watched in both directions: a sudden drop means the system got brave about things it shouldn't answer; a spike means retrieval got worse. And monthly, humans re-score a sample of the judge's verdicts — because a judge nobody audits stops being a judge.

One principle at both timescales: **a failing test blocks the merge; a failing eval freezes ingestion. The system proves it still works after every change — or the change doesn't ship.**

The System That Heals Itself

Detection is only half a nervous system. At this scale, something is *always* slightly broken: a worker dies mid-parse, a file quietly rots on disk, a connector hiccups and drops an event, an index puts on dead weight. If every small failure needs a human, the on-call engineer becomes the system's immune system — and humans make terrible white blood cells. They sleep, they take vacations, they burn out. Meanwhile small failures wait in line, and small failures that wait long enough grow up to become incidents.

So the third guiding principle, alongside simplicity and modularity, is self-healing: when something goes wrong, the system cleans and repairs itself. Humans get paged for the exceptional, never for the routine.

The repair reflexes, most of which we've already met wearing other hats:

- **A crashed worker heals by retry.** Every pipeline step is an idempotent job in the transactional queue; a dead worker's lock expires and the next worker simply picks the job up. Re-running a finished step is a no-op. Nobody restarts anything by hand at 3 a.m.
- **A corrupted file heals from snapshots.** When the weekly scrub finds a file whose hash no longer matches the registry, it quarantines the file, restores the most recent snapshot copy that *does* verify, re-checks the hash, and files a report. The human reads the report over coffee, after the repair.
- **A missed event heals by reconciliation.** The nightly three-layer count doesn't just detect gaps — it queues refetch jobs for every missing document. The daily full ACL re-sync does the same for dropped permission events. Detection and repair are one motion.
- **A degraded index heals by rebuild.** When the weekly recall check finds a partition sagging under dead vectors, that partition gets re-indexed in the next maintenance window — scheduled automatically, mentioned in the morning summary.
- **Everything derived heals by regeneration.** Wiki pages, symlink trees, summaries, chunks, whole indexes — all projections of the source of truth. The universal repair tool is gloriously dumb: throw the broken thing away and rebuild it. This is where the earlier principles pay compound interest — *parse once, everything downstream disposable* means self-healing rarely requires cleverness, just a rebuild job and patience.

Self-healing has manners, though. Retries use backoff and give up after a set number of attempts — a job that keeps failing retires to a dead-letter state and pages a human, because infinite retry is not persistence, it's a tantrum. And two things are **never** repaired by automation guessing: entitlements and regulated records. Anything ambiguous in those neighborhoods escalates to people immediately. The system heals itself; it does not *improvise* itself.

The quiet payoff: the team's time goes into making the system better, not holding it together. A platform that needs constant human attention isn't simple, no matter how few boxes are on its diagram.

Answers You Can Take to a Regulator

The generation step — the part everyone points at and calls "the AI" — is deliberately the most constrained component in the entire system. It has the least freedom of anything described in this document, and that's a compliment.

The model receives only the top-ranked, entitlement-filtered, section-expanded chunks, each wearing a name tag: document, page, version. The contract is strict:

- **Every factual claim carries a citation token**, validated by middleware against the chunks actually retrieved. An answer with an unverifiable citation is rejected before any user sees it. The model cannot cite what it was never given — the bibliography is closed-book.
- **Contradictions get surfaced, not smoothed over** — and when versions differ across time, they're presented as a timeline ("What Was True Last March?").
- **Insufficient evidence gets an honest refusal:** "*The available documentation does not contain sufficient evidence to answer this inquiry.*" The system is graded on saying this at the right moments — that's exactly what the unanswerable set is for. Saying "I don't know" well is a skill, and here it's a tested one.

And everything is written down. Every query logs the user, their evaluated roles, the applied filters, every chunk ID retrieved with its scores, the model and prompt versions, the generated answer, and the rendered citations. Monthly partitions of this log are exported nightly to **write-once storage** with hash manifests — including the chunk *text* itself, so the record stays self-contained even after some future re-chunking retires the IDs. Deletion happens through the records-management process, on the retention schedule. Not through engineering. Ever.

Three Doors for Humans

So far this design has been very generous to machines — APIs, queues, contracts, agents everywhere — and has offered humans exactly one amenity: a wiki. That won't do. A system without windows gets operated by SSH and psql, which is a fancy way of saying "an incident waiting for a typo." And a Q&A system without a place to ask questions is a philosophical exercise.

So the design includes one web frontend with three doors, one for each kind of human.

Door one: the command center. Dashboards and controls for the people who run the system:

- **Watch:** connector health and ingestion queues, index lag against the propagation SLAs, the nightly gate's trend lines (recall, citation precision, abstention correctness, groundedness), reconciliation deltas, self-healing activity (what got retried, restored, rebuilt overnight — the morning-coffee report).
- **Control:** start or pause data loads and backfills, trigger re-indexing or re-parsing for a collection, manage removals and tombstones, kick off validation runs and golden-suite tests on demand.
- **With manners:** every button drives the same documented, audited APIs used everywhere else — the command center holds no private tunnel to the database. Destructive actions demand confirmation, and everything lands in the audit log with a user identity attached. The dashboard is a *view* with steering, not a backdoor.

Door two: the admin chat. An agentic chat for operations — the command center's conversational twin. "Why did last night's gate trip?" "How far behind is the SharePoint connector?" "Re-run reconciliation for the lending collection." The agent answers by calling the same admin APIs the dashboards read, so it can *explain* as well as display — connecting an eval regression to the embedding model that shipped the day before is exactly the kind of dot-connecting agents are good at. Read operations flow freely; anything that changes state requires an explicit confirmation step; anything touching entitlements or records stays human-only, per the self-healing rules. Every agent action is audited like a human one.

Door three: the user chat. The main event — where lawyers, credit officers, and analysts actually meet the system. Ask a question, get a cited answer. Citations are clickable and open the exact page of the exact version in a document viewer. An **as-of date picker** turns time-travel queries from a power feature into a dropdown. Abstentions are displayed honestly, as answers, not apologies. And there's a **flag button** on every answer — one click sends the full trace into the evaluation review queue, which means every dissatisfied user quietly contributes to the golden suite. The complaint department feeds the immune system.

The frontend follows the house rules. It's built in **vanilla JavaScript** — no React, no frameworks, no build step. The browser already ships a perfectly good runtime; for three pages of dashboards and two chat panels, a framework is another dependency to quarantine, another supply chain to audit, and another migration in five years when it falls out of fashion. The code is **modular**: small ES modules, one per feature (chat panel, dashboard tiles, document viewer, citation renderer), each obeying the same limits as the backend — files under 800 lines, functions under 50, docs at every level, a README per directory. And **all styles live in one `styles.css`** — one file to open, one place to look, no styles hiding in JavaScript. Plain static files served from the same infrastructure, talking only to the documented APIs. The simplest frontend that fully works — which is, by now, the house style.

Monkey Business

The corpus isn't maintained by "the system." It's maintained by a group of humans — data stewards who re-scan bad batches, review low-confidence OCR, resolve deduplication conflicts, approve entity merges, and chase down the connector that's been sulking since Thursday. That work needs owners, statuses, and handoffs — otherwise it lives in email threads and in the heads of people who might be on a beach next week.

So the design gives maintainers a task system, and the tasks are called monkeys. A monkey is the next move: the task, the ticket, the thing on your back that needs to hop off. "*Re-parse the Q3 scans.*" "*Review 40 chunks below OCR threshold.*" "*Two documents claim to be the canonical version — pick one.*" Each monkey knows its owner, status, priority, and history, and carries links to the actual artifacts — document IDs, job IDs, the failed eval case, the flagged answer.

The elegant part is where monkeys come from. The system already produces them — it just didn't have a word for them yet:

- Self-healing gives up on a job after its retries → the dead-letter entry **becomes a monkey**, assigned to the right group.
- The nightly gate trips → a monkey, with the regression report attached.
- A user flags a bad answer → a monkey in the review queue, trace included.
- The scrub job finds a file it can't restore, the reconciliation finds a document the connector can't refetch → monkeys.
- And humans create their own: curation drives, collection onboarding, cleanup campaigns.

Every escalation path in this document now has an inbox, an owner, and a status. The immune system files tickets.

The human mechanics are deliberately ordinary. Each maintainer sees *their* monkeys — what's open, what's blocked, what failed overnight. Monkeys pass between people: when someone goes on vacation, their troop transfers to a teammate in one action, and nothing is orphaned. A group dashboard shows all the monkeys —

status, age, who's drowning and who's idle — because a task system where managers can't see the pileup is just a diary.

Architecturally, this is a separate FastAPI web app — its own module with its own API and its own vanilla-JS frontend (house rules: ES modules, one `styles.css`, no frameworks). Separate on purpose: the retrieval service stays lean and stable while the maintainers' workbench evolves at business speed. The monkeys themselves live in plain PostgreSQL tables — one system of record, as always — and the app talks to the rest of the platform only through the documented APIs.

And because the workbench evolves at business speed, **maintainers get to build their own tools — by vibe-coding.** A steward who needs a review screen for last month's low-confidence scans, or a one-page dashboard for a cleanup campaign, describes it and lets an AI assistant generate it — a small vanilla-JS page over the existing APIs, shipped in an afternoon, no platform-team ticket required. The same goes for **workflows**: maintainers create and schedule data jobs — extraction, cleaning, transformation, loading, verification — composed from idempotent steps and run through the same transactional job queue as everything else.

Vibe-coding gets guardrails, not a leash. Generated tools and workflows live inside the maintainers' app, consume only the documented APIs (so ACLs and audit apply automatically — a vibe-coded page cannot see what its author can't), and pass the same CI as human-written code: the tests, the size limits, and the AI architecture-conformance review. Fast where speed is cheap, gated where mistakes are expensive. The business gets its tools in hours; the platform keeps its seams.

The Day Everything Goes Wrong

Now the section nobody enjoys writing. Suppose the worst — all of it, preferably on a Friday: a bad ACL sync poisons entitlements, an operator fat-fingers a `DROP TABLE`, ransomware arrives, an availability zone catches fire. What survives?

First, an unpopular truth: **replicas are not backups.** A standby replica will faithfully replay your `DROP TABLE` within milliseconds — that's its job, and it's very good at it. Replication protects against hardware failure. Backups protect against mistakes and malice, which have no SLA.

So: the database ships every write to an archive continuously (point-in-time recovery to any second — losing at most one hour is the written objective), plus weekly full and daily incremental base backups. Backup storage lives in a **separate account with independent credentials**, so no single compromised admin can torch both the data and its safety net in one evening. Volume snapshots — consistent by construction, thanks to the write-once discipline — are copied cross-AZ and **locked** against deletion: even ransomware holding admin keys can't erase the past. The audit archive is write-once by design.

Restores follow a scripted order, because the database and the volume must agree with each other afterward: **database first**, to the chosen moment; **volume snapshot from at-or-after** that moment (write-once means a newer snapshot only carries harmless extras, while an older one might be missing referenced files — newer is always safe, older never is); then a reconciliation pass that re-hashes every referenced file and queues re-fetches for gaps. **ACLs re-sync before the API reopens** — restoring stale entitlements would be a permission leak with a runbook as the root cause, which is not a sentence anyone wants in a postmortem.

And because a backup that's never been restored is a rumor: **a monthly automated restore test** rebuilds the database in an isolated environment, verifies integrity, and runs the golden suite against it — a failed restore pages on-call like a production outage. Quarterly, a full disaster-recovery drill runs the whole sequence against a stopwatch, and the results go to Compliance and Business Continuity in writing.

Recovery targets, stated plainly: lose at most one hour of data; be back within eight hours for a full-site event, faster for lesser disasters.

How Big Is This, Really?

The numbers that make the single-database bet rational rather than reckless:

On the document volume:

What	Size
Originals (3M × ~1 MB)	~3 TB
Parsed derivatives (JSON + Markdown)	~1–2 TB
Rendered wiki tree	~0.3–0.5 TB
Headroom for growth	~2–3 TB
Provisioned	**8–10 TB**

In PostgreSQL:

What	Size
Chunk text (20–25M chunks)	~40–60 GB
Vectors + HNSW index (half-precision)	~50 GB
Full-text (BM25/GIN) indexes	~20–30 GB
Registry, ACLs, summaries	~15–25 GB
Document graph	~5–10 GB
Audit log growth	~2–5 GB/month
Steady state	**~200–300 GB**

A single instance with 128–256 GB of RAM keeps the vectors and hot indexes in memory, where they belong. Backups run 2–3× database size; the first volume snapshot equals used bytes, with small daily deltas after.

Speed targets: **p95 under 1.5 seconds** for retrieval, under 4.5 end-to-end. Ingestion: the full backfill lands in weeks — bulk-loaded into unindexed tables with the indexes built once at the end, because building indexes *during* a 25-million-row load is how a backfill becomes a quarter-long hostage negotiation. After that, incremental sync in minutes.

Don't Chase the Latest Version

One attack vector remains, and it's the one most teams leave wide open — enthusiastically, on purpose, every day.

The whole design keeps data inside the firm's walls: self-hosted models, mounted volumes, private endpoints. And then a developer types `pip install`, and the build cheerfully reaches out to a public registry and downloads whatever was uploaded yesterday. By anyone. Modern supply-chain attacks live exactly there: a hijacked maintainer account ships a poisoned point-release; a typosquatted package waits patiently for one mistyped name; a backdoor hides in a build script — the 2024 `xz` incident came within weeks of shipping inside every major Linux distribution. The freshest package is the least reviewed package. "Latest" is not a version. It's a gamble.

The solution: every dependency goes through quarantine — pinned, aged, and verified. Like new hires, packages need a probation period.

Python itself comes from Homebrew (macOS or Linux), pinned to a major.minor line and upgraded on purpose, never by accident:

```
brew install python@3.12
```

Environments and modules are managed with `uv`, under two standing rules: nothing published in the last 30 days, and no prereleases. Let the rest of the world do the beta testing — they volunteer daily. In `pyproject.toml`:

```
[tool.uv]
exclude-newer = "30 days"    # ignore anything published in the last month
prerelease = "disallow"      # never alphas, betas, or release candidates
```

Upgrades become an explicit, reviewable act:

```
uv lock --upgrade
uv sync
```

Or, for a `requirements.txt`-based flow:

```
uv pip install --upgrade   --exclude-newer "30 days" -r requirements.txt
uv pip install --reinstall --exclude-newer "30 days" -r requirements.txt
```

The lockfile is committed, so every environment — laptop, CI, production worker — resolves to byte-identical, properly-aged packages. No surprises, which is the highest compliment in operations.

PostgreSQL follows the same rule. Development machines run a pinned major line from Homebrew (`brew install postgresql@16`); production uses the official PGDG repositories with the version held explicitly. Minor releases get adopted a few weeks after they ship, once the ecosystem has kicked the tires. A brand-new major — the dreaded `.0` — never goes straight to production: wait for the first minors, and rehearse the upgrade on the monthly restore-test instance, which conveniently exists anyway.

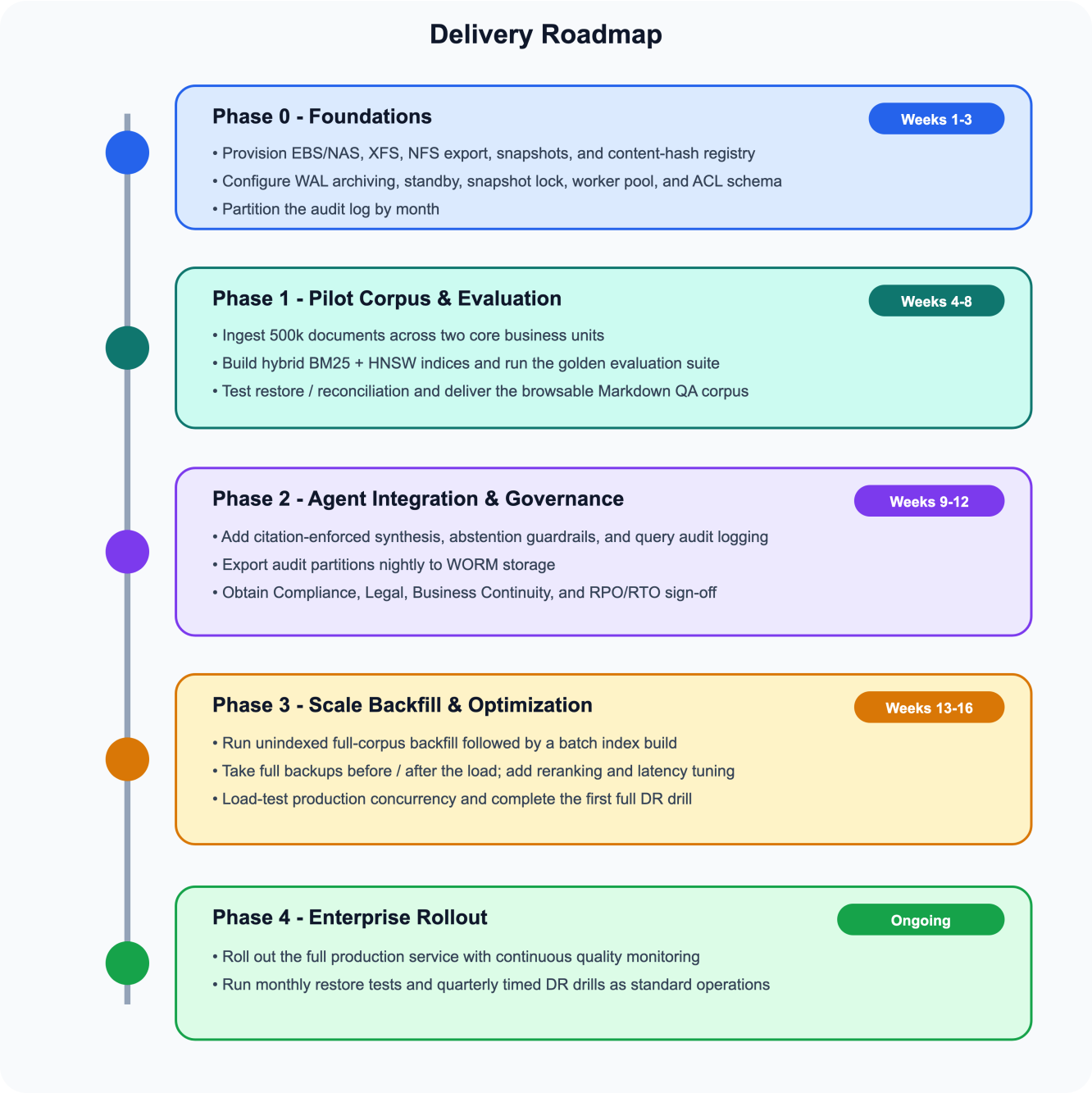
Docker images get the strictest treatment, because a tag is a promise nobody has to keep:

- Official images only (`postgres`, `python`) — never the look-alikes.
- Pinned **by digest**, not by tag — `postgres:16.6@sha256:…` — so the image cannot change underneath you. `:latest` is banned outright.
- Pulled once into the firm's **internal registry**, scanned (Trivy/Grype) on the way in, and only then available to builds. Production hosts pull from the internal registry, never from the public one.

The system guarding three million confidential documents should not be one `pip install` away from running last night's upload. So it isn't.

The Road from Here

The build order is chosen so that trust is earned before scale is attempted — prove it, then grow it:



Phase 0 (weeks 1–3) lays the foundations: storage, database, WAL archiving, ACL schema. **Phase 1 (weeks 4–8)** ingests a 500k-document pilot across two business units, builds the hybrid indexes, extracts links and effective dates, renders the first wiki — and runs the golden suite for the first time, while curators grade the extraction on the same corpus they're QA-ing anyway. **Phase 2 (weeks 9–12)** adds the guarded generation layer — citations, abstention, audit logging, WORM export — plus graph expansion, as-of retrieval, and the nightly evaluation gate, ending in Compliance/Legal/BC sign-off. **Phase 3 (weeks 13–16)** is the full backfill and load testing, bracketed by base backups and closed with the first timed DR drill. **Phase 4** is enterprise rollout — at which point the nightly gates, restore tests, and drills stop being milestones and simply become weather.

v2 candidates wait patiently behind evaluation evidence, in the spirit of the opening bet: LLM-based relationship extraction on high-value collections, thematic clustering with summary pages, and any migration past a scaling gate.

The Shape of the Thing

Strip away the details, and the whole design is eight decisions:

1. **Simplicity above all.** The fewest moving parts that still deliver the functionality. Simplicity is what makes the project do-able, flexible, maintainable, and affordable — buildable by a small team, prototyped in weeks, run without a fleet of clusters or heavy hardware. Complexity is the project's primary risk, admitted only through measured gates.
2. **Modularity keeps the bets reversible.** Every capability sits behind a contract — retrieval behind an API, sources behind connector plugins, parsers behind the derivative format, models versioned per artifact — so any part can be changed, replaced, or removed without understanding the whole. Loose coupling, clean seams: separate dumplings, never a clump.
3. **One system of record.** PostgreSQL holds truth — content indexes, entitlements, graph, queue, audit — so consistency is a transaction, not a distributed-systems project.
4. **Parse once; everything downstream is disposable.** Originals and derivatives are permanent; chunks, vectors, indexes, and wiki are projections, rebuildable at will. This is what makes every future migration boring — and boring migrations are the good kind.
5. **Search three ways** — meaning, keywords, structure — with time as a dimension, because each method catches what the others miss.
6. **Security and provenance are load-bearing.** ACLs filter before ranking; citations verify before display; every answer leaves an immutable trail; saying "I don't know" is a graded skill.
7. **Trust is re-earned continuously.** A pyramid of tests gates every merge; evaluation gates, reconciliation counts, hallucination sampling, and restore drills gate every day. The system proves it still works after every change — or it stops and says so.
8. **Self-healing by design.** Crashed jobs retry, corrupted files restore from verified snapshots, missed events refetch, degraded indexes rebuild, and every derived artifact can be regenerated from the source of truth. Humans are paged for the exceptional, never the routine — and nothing touching entitlements or records is ever repaired by guessing.

A corpus of three million documents, one honest database, and a system designed to be *caught* being wrong before a user ever is — and to patch itself up before anyone has to ask. That's the design.

Appendix A: Requirements Reference

A.1 Functional

ID	Requirement
F1	Ingest documents from multiple enterprise sources (file shares, SharePoint, email archives, document management systems) in PDF, DOCX, PPTX, XLSX, TXT, HTML, Markdown, and common scanned image formats.
F2	Handle scanned documents via OCR, with extraction confidence scored and attached at the chunk/page level.
F3	Preserve document hierarchy: section headers, page and slide numbers, table schemas, footnotes, and speaker notes.
F4	Detect and handle exact duplicates, near-duplicates, and superseded versions with alias mapping.

ID	Requirement
F5	Support hybrid retrieval — exact/lexical match (tickers, ISINs, clause numbers, dates, party names) **and** semantic match (conceptual questions, thematic synthesis, paraphrase).
F6	Filter by rich metadata: source, business unit, document type, date range, classification, and language.
F7	Return answers with verifiable citations at the page / slide / section / table level, deterministically linked to the exact document version used.
F8	Abstain or flag uncertainty when evidence is weak, contradictory, or superseded.
F9	Expose retrieval as an independent, secured REST/gRPC service consumable by the agent and other internal systems.
F10	Re-index automatically upon source modification; tombstone deleted content promptly.
F11	Provide a human-browsable, normalized Markdown corpus for domain expert QA, curation, and compliance audits.
F12	Maintain a typed inter-document link graph (supersession, amendment, citation, exhibit, shared entities) and an entity index in PostgreSQL, populated deterministically at ingestion and usable for retrieval-time context expansion.
F13	Render the corpus as an interconnected wiki: Markdown files with YAML frontmatter and `[[wikilinks]]` (Obsidian-compatible), generated from the database, human-navigable and grep-friendly, scoped to authorized audiences.
F14	Support point-in-time ("as-of") retrieval via an optional `as_of_date` parameter, with historical answers explicitly flagged.
F15	Provide human interfaces: an operations command center (dashboards and controls for data loads, removals, updates, validation, and testing), an agentic admin chat, and an agentic end-user chat with clickable citations, as-of date selection, and one-click answer flagging into the evaluation queue. All interfaces consume the same audited APIs; state-changing actions require confirmation and are logged.
F16	Provide a maintainer task system ("monkeys") as a separate FastAPI web app: per-user task lists with status and failure visibility, reassignment and vacation handoff within groups, a group-wide dashboard, automatic task creation from dead-letter jobs / eval-gate trips / flagged answers / scrub and reconciliation failures, plus AI-assisted (vibe-coded) mini-tools and schedulable ETL workflows — all consuming only documented APIs and passing the standard CI and conformance gates.

A.2 Security, Compliance, and Governance

ID	Requirement
S1	Access control strictly mirrors source-system entitlements. ACL filters are applied **pre-ranking** at the database layer; unauthorized users can never retrieve or observe chunks from restricted documents.
S2	Information barriers (ethical walls, client/deal segregation) enforced at the tenant / collection partition level.
S3	Automated classification tags at ingestion: PII, MNPI, client-confidential, regulatory, and retention category.
S4	Full audit log per query: user identity, evaluated roles/groups, applied filters, retrieved chunk IDs, ranking scores, model/prompt versions, generated response, and rendered citations.
S5	Complete data residency: originals, derivatives, vector indices, and audit logs remain within the firm's approved security boundary. Models are self-hosted or private enterprise endpoints.
S6	Release v1 is strictly **read-only**: no execution of financial transactions, external communications, or automated decisions.
S7	Prompts, retrieved contexts, and outputs are archived as immutable business records; closed audit-log partitions are exported to write-once storage.

A.3 Non-Functional

ID	Requirement
N1	**Corpus Scale:** Low single-digit millions of source documents; 20–30 million indexed chunks after deduplication.
N2	**Query Latency:** p95 retrieval under 1.5 s (excluding LLM generation).
N3	**Ingestion:** Backfill in weeks; incremental sync in minutes.
N4	**Operational Simplicity:** Zero multi-cluster maintenance overhead for v1.
N5	**Reliability:** Transactional, observable, crash-resilient job processing with automatic retries.
N6	**Measurable Quality:** Recall@20, citation accuracy, groundedness, and permission-leak rate continuously evaluated against a golden dataset.
N7	**Portability:** Chunk indices are disposable and 100% reconstructible from stored derivatives without re-parsing raw files.
N8	**Storage Platform:** Originals and derivatives on mounted block/file volumes inside the firm's network — no object-store dependency.
N9	**Backup & Recovery:** RPO ≤ 1 hour, RTO ≤ 8 hours; immutable backups inside the security boundary; automated restore tests at least monthly.
N10	**Lifecycle & Continuous Evaluation:** Content changes visible within minutes; ACL revocations within 5 minutes; nightly evaluation gate verifies quality, completeness, and hallucination level after each day's updates and freezes ingestion on regression.

Appendix B: Technology Choices

Layer	Selected Technology	Why
Originals & derivatives	Mounted volumes (EBS/NFS or NAS/SAN), XFS + LVM, content-addressed and write-once	In-boundary, snapshot-consistent by construction, POSIX-fast for parsers
Parsing	Docling (heavy path) + PyMuPDF (fast path)	Layout/table fidelity with OCR confidence; 10× faster path for born-digital files
Database & indexing	PostgreSQL 16+ with `pgvector` (HNSW, `halfvec`) and `pg_search` (BM25)	One engine for ACL joins, keyword search, vector search, queue, and audit
Document graph & wiki	Plain PostgreSQL edge/entity tables; wiki rendered as Markdown + frontmatter + wikilinks (Obsidian-compatible)	Structure stays in the system of record; wiki is a disposable projection
Job queue	PostgreSQL `SKIP LOCKED`	Transactional, broker-free
Embeddings	Self-hosted open models (BGE / E5 / Nomic class) on local GPUs	Data boundary; no per-token fees across 10B+ tokens
Reranker	BGE-Reranker-Large cross-encoder; LLM rerank only as evaluated escalation	Precision on multi-hop clauses within the latency budget
Retrieval API	Python FastAPI / asyncpg	Lightweight, typed, decoupled from agents. Python over Rust: the heavy lifting runs in C/GPU libraries (Postgres, CUDA, PyMuPDF), the parsing/ML ecosystem is Python-first, and any measured CPU-bound module can later be swapped to Rust behind its contract

Layer	Selected Technology	Why
Frontend (command center, admin chat, user chat)	Vanilla JavaScript ES modules — no frameworks, no build step; all styles in a single `styles.css`; static files served from existing infrastructure; consumes only the documented, audited APIs	Simplicity: no framework dependency to quarantine or migrate; modular per the code rules (one module per feature, README per directory); ACLs and audit apply identically to humans and agents
Maintainer workbench ("monkeys")	Separate FastAPI app + vanilla-JS frontend; monkey tables in PostgreSQL; auto-created from dead-letter/eval/flag/scrub events; vibe-coded mini-tools and scheduled ETL workflows running through the shared job queue, gated by the standard CI + AI conformance review	Retrieval service stays lean while the workbench evolves at business speed; every escalation path gets an owner, a status, and a handoff mechanism
Toolchain & dependencies	Homebrew-installed Python (pinned major.minor); `uv` with `exclude-newer = "30 days"` and `prerelease = "disallow"`; PGDG-pinned PostgreSQL; Docker images by digest via scanned internal registry	Supply-chain safety: aged, pinned, verified dependencies; no `:latest`, no day-old packages
Testing	pytest (unit + module tests via public APIs with faked ports); containerized PostgreSQL for integration tests; fixture corpus for pipeline tests; `import-linter` + size/docstring checks in CI; AI-agent architecture-conformance review gating merges	Every layer guarded: functions, module contracts, real adapters, answer quality (golden suite), and the architecture rules themselves
LLM inference	Private enterprise endpoint / self-hosted vLLM	Inside the VPC perimeter
Backup & recovery	pgBackRest PITR + standby; locked cross-AZ snapshots; WORM audit exports	One-hour RPO; ransomware-resistant; restore path proven by drills

Appendix C: Risk Register (Condensed)

Risk	Mitigation
Permission leakage (incl. via graph links or wiki)	Pre-ranking SQL ACL joins everywhere including graph hops; wiki rendered only in curator enclave; adversarial leak tests in CI; 0.00% tolerance
Missing specific clauses	Adaptive routing lets identifier queries bypass summary filtering
Stale or wrong-period answers	Validity windows, as-of filtering, temporal/currency ranking boosts, supersession flags, as-of golden cases
Hallucinated citations	Middleware validates every citation token against retrieved chunks; unanswerable-set testing; nightly judge sampling
Silent ingestion failure / stale index	Nightly three-layer reconciliation; index-lag metric; self-retrieval probes
Routine transient failures (crashed workers, corrupt files, missed events, index bloat)	Self-healing: idempotent retries with backoff, automatic quarantine-and-restore from verified snapshots, reconciliation-driven refetch, automatic partition reindex, regeneration of derived artifacts; dead-letter + page only after retries are exhausted; entitlements and records always escalate to humans
Quality regression after updates	Nightly golden-suite gate vs. 7-day baseline; page + ingestion freeze on trip

Risk	Mitigation
Architecture erosion in code	`import-linter` + size/docstring checks in CI; AI-agent conformance review on every merge
Vibe-coded tools bypassing controls	Generated tools live in the maintainers' app, reach data only through documented APIs (ACLs + audit apply automatically), and pass the same CI, size, and conformance gates as human-written code
Document volume loss/ corruption	Locked cross-AZ snapshots; write-once discipline; hash scrub
PostgreSQL loss / logical corruption	Continuous WAL archiving + PITR; standby for failover; monthly restore tests
Audit record loss	Monthly partitions exported nightly to WORM with hash manifests; records-policy-only deletion
Backup destruction (ransomware / rogue admin)	Snapshot lock, separate backup account, WORM archive, scrub-verified clean-point restore
Untested restore path	Monthly automated restore tests page on failure; quarterly timed DR drills reported to Compliance/BC